

Smart Parking Monitor — Dashboard Requirements

Version: v0.1 (MVP)

Author: Hongyi Mei

Date: 2026-07-29

Audience: Dashboard backend + frontend

0. Summary

Three cameras watch a single parking spot. The screen shows one tile: red when a car is present, green when the spot is free, with 2/3 underneath meaning two of the three votes said occupied. Above the tile is how long the car has been parked.

The lower half of the page is a timeline of the past 10 hours, showing when the spot was occupied and when it was free, alongside a list of each parking session's start time, end time, and duration.

At the bottom, three rows — one per camera node — show online or offline status and the time of each node's last report. Unplug a node and its row greys out, and the vote count above changes from 2/3 to 2/2.

The details follow.

1. Background

Three ESP32-S3 camera nodes run occupancy detection on the same parking spot and each report their verdict over MQTT. The gateway (Raspberry Pi) takes a majority vote across the three reports to produce the authoritative occupancy state, and writes it to SQLite.

Your scope: everything downstream of SQLite — reading the database, the API, and the frontend page.

Not your scope: MQTT, node firmware, and the fusion algorithm. As far as you are concerned the database is read-only.

Majority voting is used because a single vision node misclassifies. Measured accuracy is about 85% for a single node and about 94% after three-node fusion. The UI therefore needs to distinguish a unanimous 3/3 vote from a narrow 2/3 one.

2. Demo Scenario

The demo uses a **single parking spot (Spot A)**. The flow is roughly:

- The spot is empty -> the page shows green / vacant.

- Place a model car in the spot -> within a few seconds the page turns red / occupied and starts timing.
 - Unplug the power on one node -> the page flags that node offline and the vote drops to two nodes.
 - Remove the car -> the page returns to vacant and the session duration appears in the history list.
-

3. Functional Requirements

3.1 Live Status (required)

- Spot A is either **occupied** or **vacant**, clearly distinguished by color.
- Show the vote count for the current round, e.g. 2/3 or 3/3.
- When the denominator is below 3 (a node has dropped out), show a visual indication — it must not be confusable with the normal state.
- If currently occupied, show the **elapsed parked time** (from the start of this occupancy to now).
- A 3–5 second refresh is fine. Polling or SSE are both acceptable; WebSocket is not required.

3.2 History View (required)

- Show occupancy changes for Spot A over the **past 10 hours**.
- The form is up to you — timeline band, line chart, or table — as long as it is clear when the spot was occupied and when it was free.
- Include a **parking session list** for the window: start time, end time, duration.

3.3 Node Health (required)

- List all 3 nodes with **online / offline** status for each.
- Show each node's last report time.
- Offline nodes need to be clearly marked.

3.4 Explicitly Out of Scope (not in MVP)

- Billing, rates, payments
 - Timeout alerts, violation detection
 - User registration, vehicle identification, license plates
 - Multi-spot management
 - Login / authentication
 - Mobile layouts (desktop browser is sufficient)
-

4. Data Source

The SQLite file lives on the Raspberry Pi (path to be confirmed at deploy time). WAL mode is enabled, so your reads will not be blocked by writes.

See `schema.sql` for the table definitions. A few usage notes:

Requirement	Data source
Live status	Latest row of the fused table by <code>ts</code>
Vote count	<code>fused.votes_occ / fused.votes_total</code>
History view	fused table, queried over a <code>ts</code> range
Parking sessions	sessions table; <code>ended_at IS NULL</code> means in progress
Elapsed parked time	Current time - <code>sessions.started_at</code>
Node health	nodes table (current status + last report time)

The `readings` table holds the raw reports from the three nodes. It **does not need to be displayed** in the MVP; it is kept for later analysis of single-node reliability.

All timestamps are unix epoch seconds (UTC); the frontend is responsible for converting to local time for display.

5. Technical Conventions

- Backend language and framework are your call — Flask or FastAPI are both fine.
 - No frontend framework required; plain HTML/JS is enough.
 - Deployed on the Raspberry Pi itself, accessed over the LAN by IP.
 - Open the database connection read-only (`file:...?mode=ro`) to avoid accidental writes.
-

6. Open Questions

- Final path of the database file (to confirm once the gateway is deployed)
 - Exact presentation of the history view (sketch it and we'll compare notes)
 - Whether you want a `.db` file with fake data for local development
-

7. Timeline

No rush. Suggested order: get 3.1 live status working end to end first, then fill in 3.2 and 3.3.